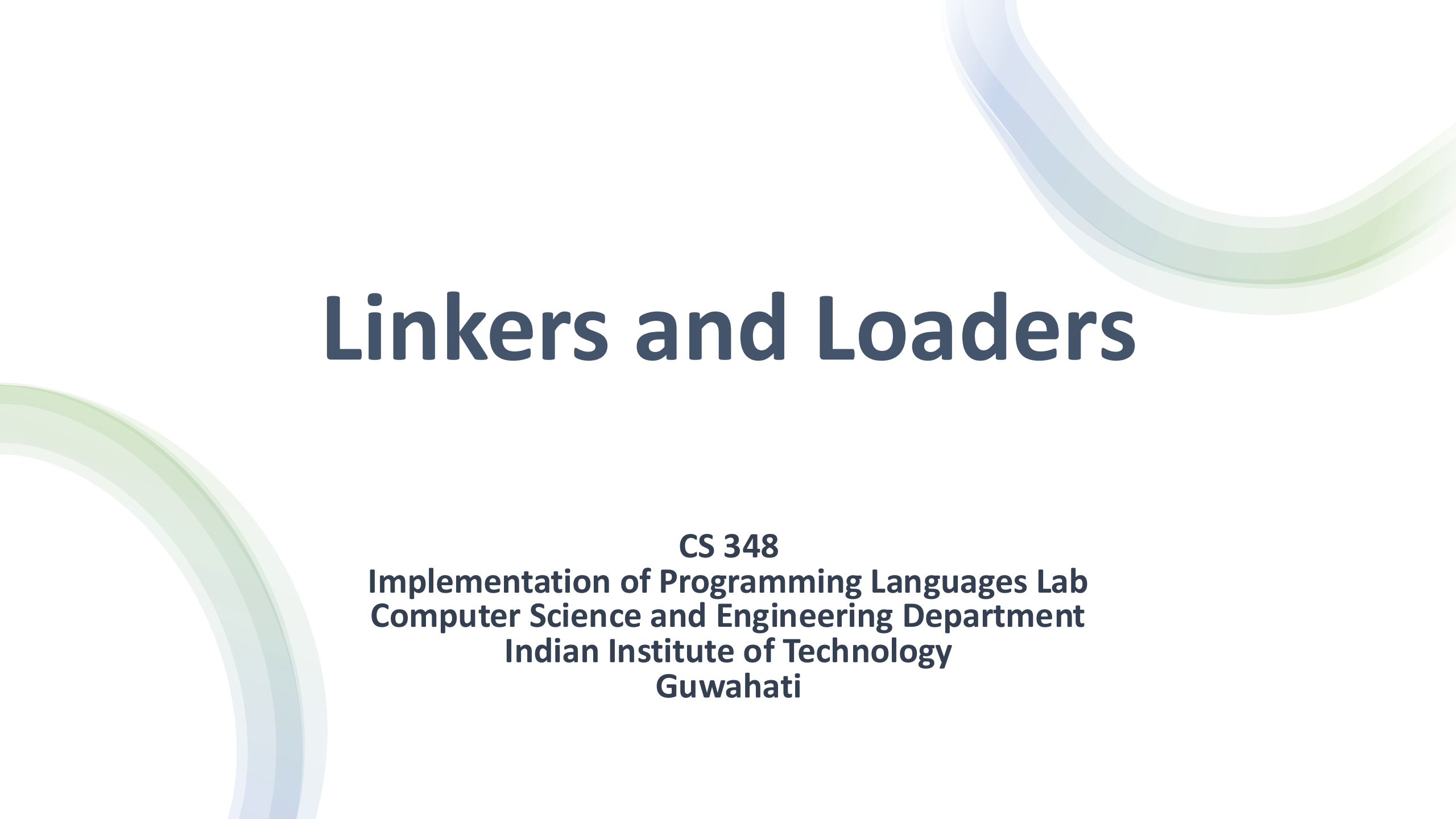




Linkers and Loaders

CS 348

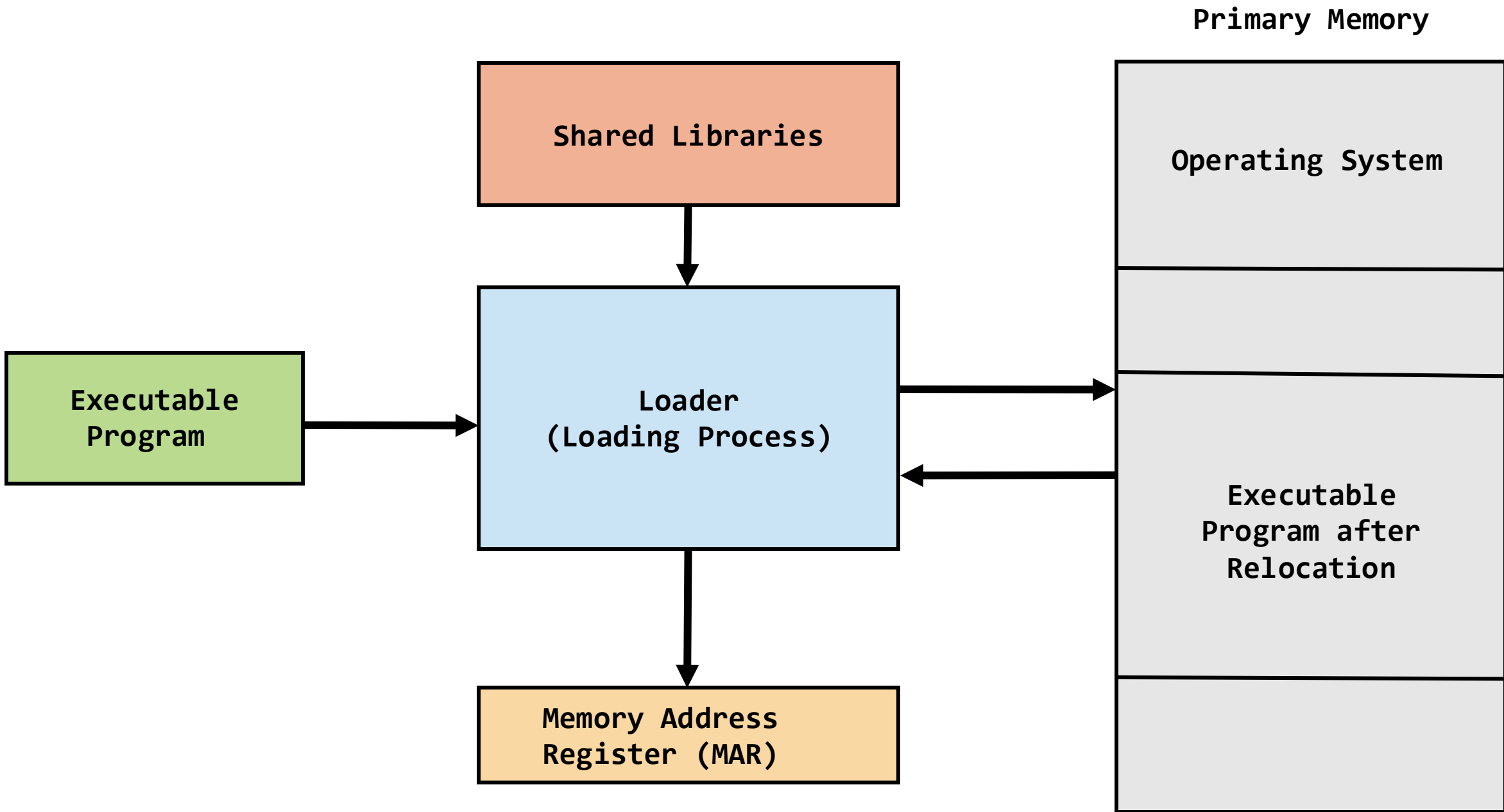
**Implementation of Programming Languages Lab
Computer Science and Engineering Department
Indian Institute of Technology
Guwahati**

Contents

- Overview of Loader
- Basic Principles
- Functions
- Types of Loaders

Loader

- A loader is a system software program that performs loading functions
- It brings object programs into memory and start its execution



Basic Loading Principles

A loader performs two fundamental operations:

- Program Preparation
- Control Transfer

These ensure that:

- The program is correctly placed in memory from secondary storage
- All addresses are valid
- Execution can safely begin

Preparing Program for Execution

Before transferring control to a program, the loader must transform the executable file into a fully prepared execution image in memory.

- Memory Allocation
- Relocation
- External Symbol Resolution
- Data Segment Initialization
- Stack and Heap Setup
- Register and Environment Initialization

Memory Allocation

Reserve sufficient contiguous memory space for the program.

Loader reads the header of the executable and determines:

- Size of text (code) segment
- Size of initialized data
- Size of uninitialized data (BSS)
- Required stack size

Considerations:

- Avoid overlap with Operating system and other processes.
- Must satisfy other alignment constraints
- May require page aligned allocation.

Requests memory from OS memory manager.

Base Address (Program Load Address – PLA) is assigned after these considerations.

Program Load Address (PLA)

- PLA (Program Load Address) is the starting memory address where the program will be loaded.
- It acts as a base address for all memory placement.

Why PLA is Important?

- Determines where text and data segments begin
- Used in relocation calculations.
- Ensures no memory overlap with OS or other programs.

Relocation

Object programs often contain relative addresses.

What Must Be Relocated:

- Absolute addresses in instructions
- Jump targets
- Data references
- Address constants

Without relocation:

- Program would branch to incorrect memory locations.

Resolving External References (Linking Phase)

Programs may reference:

- External functions
- Library routines
- Global variables defined in other modules

These references must be resolved before the final executable is loaded into memory.

The loader must:

- Locate the symbol in symbol table or shared library.
- Determine its absolute memory address.
- Replace placeholder reference with real address.

Initializing Data Sections

Executable typically contains:

- Text Segment (Code)
- Initialized Data Segment
- Uninitialized Data Segment (BSS)

Loader Actions:

- Copy text segment into memory.
- Copy initialized data values.
- Allocate space for BSS and initialize to zero.

Stack and Heap Setup

Stack Initialization

- Allocates stack region.
- Initializes Stack Pointer (SP).
- Places program arguments on stack.
- Sets up return address frame.

Heap Initialization

- Defines start of heap.
- Prepares dynamic memory management.
- Heap grows dynamically during execution.

Register Initialization and Environment Setup

Before execution loader must initialize:

- Program Counter (PC) → Entry point
- Stack Pointer (SP)
- Base registers (if used)
- Global pointer registers (in some architectures)

In modern OS loaders:

- Environment variables are passed.
- Command-line arguments are prepared.
- Dynamic linking tables are set.
- Runtime startup code is invoked.

Sequence: Loader → Runtime Startup Routine → main()

Final Verification Before Execution

Before transferring control, loader ensures:

- All relocation entries processed.
- No unresolved external symbols.
- Memory boundaries respected.
- Entry point exists and valid.
- Execution permissions set properly.

If any error occurs during this process:

- Loader aborts execution.
- Reports loading error

Execution Image

After the preparation stage:

- Code is correctly placed.
- Data is initialized.
- Symbols are resolved.
- Stack prepared.
- Registers initialized.
- Entry point defined.

The program now exists in memory as a **self-contained execution image**.

Transfer of Control to Program

Final loader step:

- Determine entry point address.
- Load entry address into:
 - Program Counter (PC)
 - or Memory Address Register (MAR)
- Execution begins from entry point.

This marks transition:

Loader → User Program

Recap of Loader Procedure

- Read the header of the executable file to find out the size of the text and data.
- Create an address space which is large enough for the program.
- Copy the instructions and data into primary memory.
- Copy parameters to the main program onto the stack.
- Initialize the machine registers and set the stack pointer.
- Jump to a start-up procedure that copies the parameters into the argument registers and calls the main procedure.

Types of Loaders

The major loader types are:

- Assemble-and-Go Loader
- Absolute Loader
- Relocating Loader
- Direct Linking Loader

These differ based on:

- Relocation capability
- Linking capability
- Flexibility

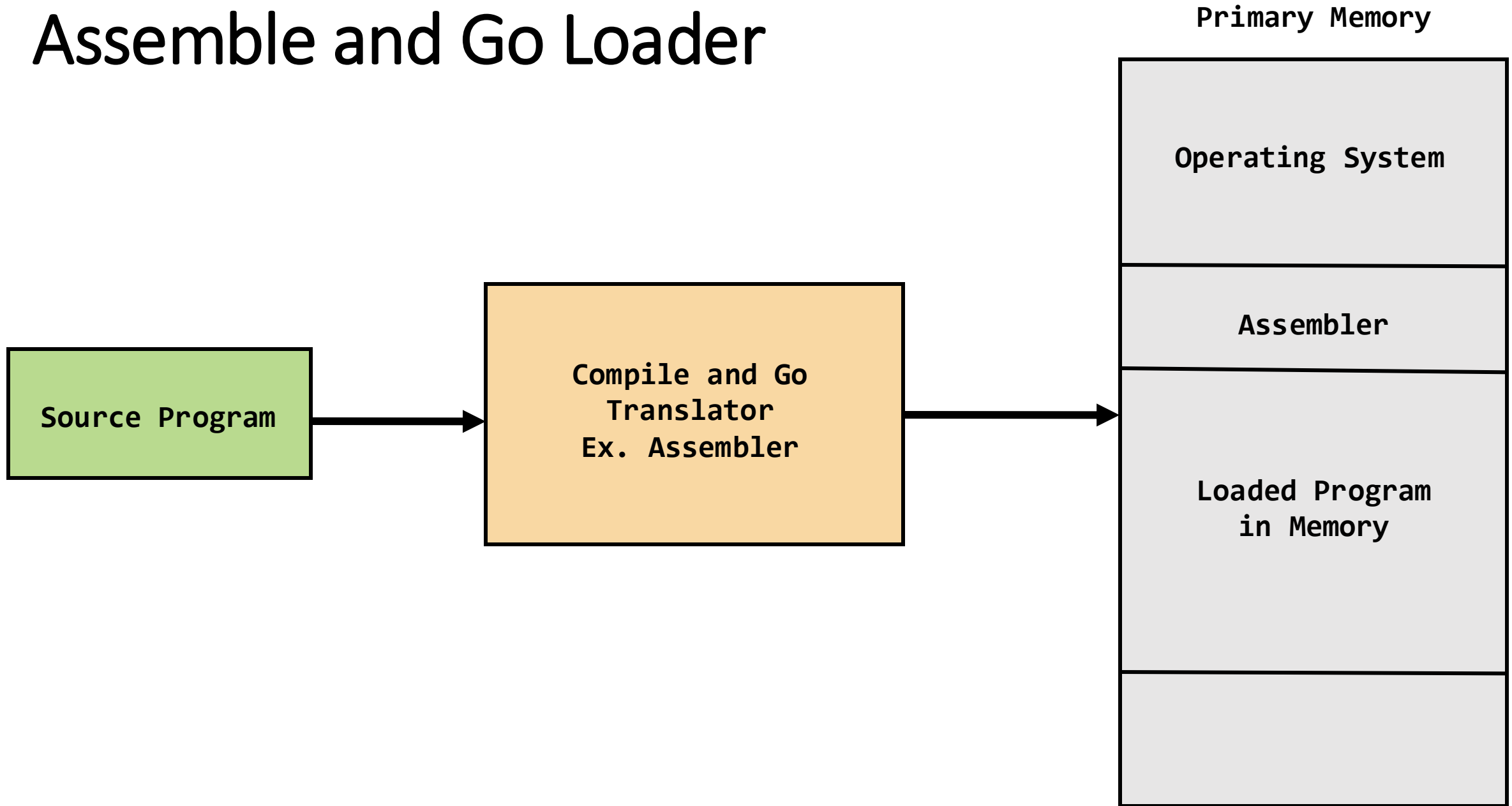
Assemble and Go Loader

- Simplest loader mechanism
- Assembler translates and loads simultaneously
- No separate object file
- No relocation performed

Source → Assembler → Memory → Execute

- The typical example is WATFOR-77, a FORTRAN compiler which uses “load and go” scheme

Assemble and Go Loader



Assemble and Go Loader

Advantages:

- Simple implementation
- No separate loader needed i.e no additional routines are required to load the assembled code into memory.

Disadvantages:

- Wastes memory (assembler stays in memory)
- Program retranslated every run
- Cannot easily support multi-module systems
- Execution time is high depending on complexity of source program

Absolute Loader

- Loads program at **fixed address**
- No relocation performed
- Starting address specified during assembly

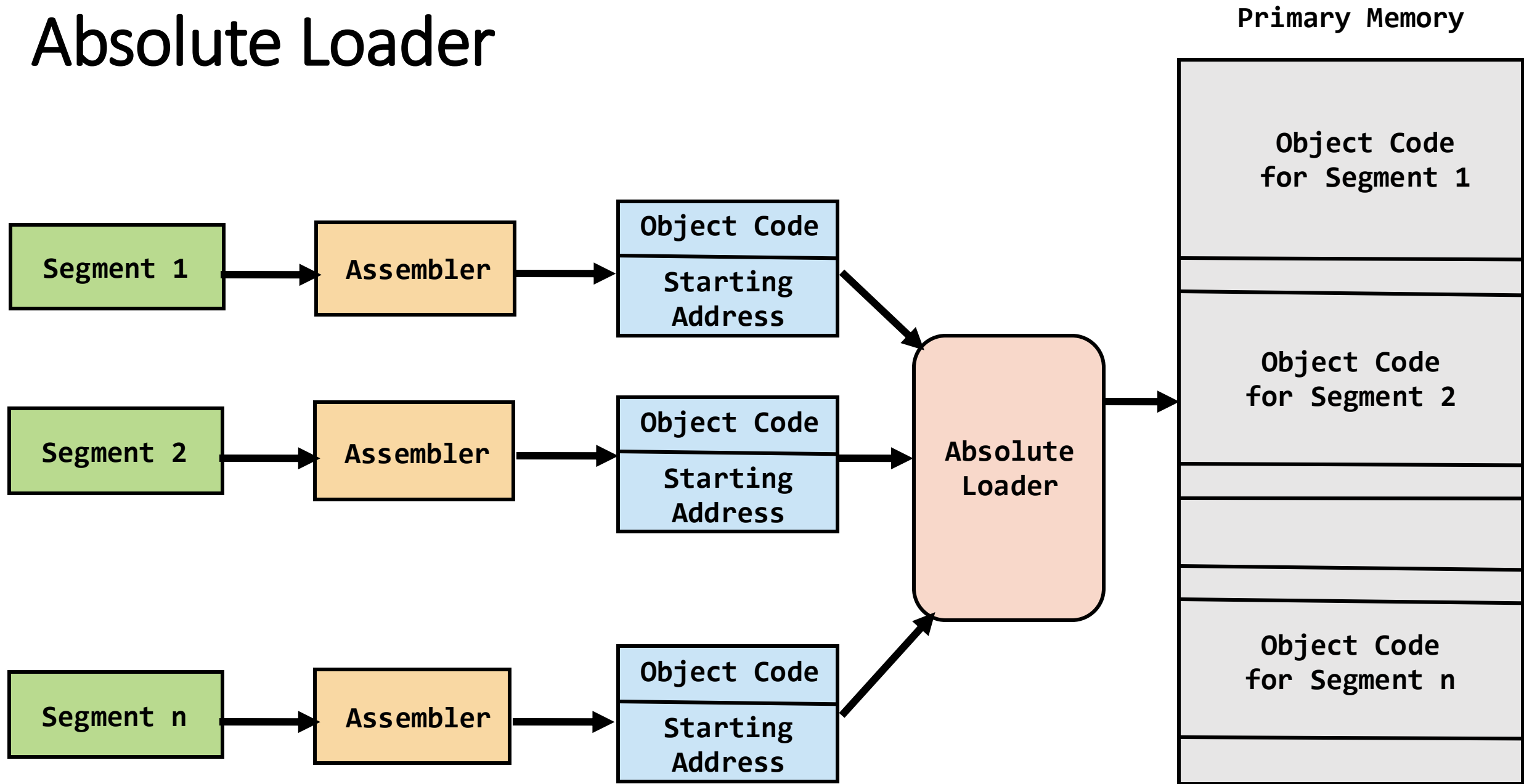
Absolute loader reads record having:

- Starting address
- Length
- Text (machine code)

T 100 03 141033

If memory location needs to be changed, program must be reassembled.

Absolute Loader



Absolute Loader

Advantages:

- Simple
- Fast execution

Disadvantages:

- Programmer must manage addresses including external references or linking of different subroutines
- Programmer should have knowledge of memory management.
- No flexibility
- Hard to maintain large programs

Relocating Loader

- Load program at any available memory location
- Performs relocation automatically
- Uses relocation information from assembler
- Execution of the object program is done using any part of the available and sufficient memory.
- Assembler assembles each procedure segment independently

Relocation is done by using Relocation Bit and Transfer Vector

Relocating Loader

Relocation bit:

- It is used for relocation, using a bit that is present in object code.
- This bit is associated with each instruction.
- If its 1, then relocation is done, else not done.

Example:

Instruction	Address	Relocation Bit
LOAD 100	05	1
ADD R1, R2	--	0

Relocating Loader

Transfer Vector

- Used to resolve external references
- Stores external subroutine names
- Calls initially point to transfer vector
- Loader fills actual address

Initially when we call the subroutine, it transfers to the corresponding location of transfer vector.

- Finally, loader calculates absolute location (exact position).
- It is used on computers with a fixed length direct address instruction format

Relocating Loader

Advantages:

- Flexible memory placement
- No need to reassemble when memory location changes

Disadvantages:

- More complex
- Slightly slower due to relocation processing

Direct Linking Loader

Direct Linking Loader is the most common type of loader

- Relocatable loader.
- Does not have direct access to the source code.

Placement of object code in memory is done,

- Either the address of object code could be Absolute which can be directly placed in memory
- If the address is relative, then the assembler informs the loader about the relative address.

Direct Linking Loader

Information from the assembler required by the loader,

- The length of the program
- A list of external symbols
- Information about address constant
- Machine code translation of the source program

Loader Actions

- Build External Symbol Table (ESTAB)
- Assign memory locations
- Replace CALL reference with actual address
- Perform relocation
- Start execution

Direct Linking Loader

Advantages:

- Modular programming
- Supports large systems
- Flexible

Disadvantages:

- More complex
- Loader occupies memory

Direct Linking Loader

Loader Type	Relocation	Linking	Flexibility
Assemble-Go	No	No	Very Low
Absolute	No	Manual	Low
Relocating	Yes	Partial	Medium
Direct Linking	Yes	Yes	High